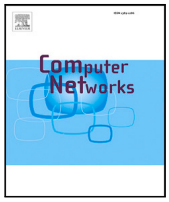
Computer Networks 250 (2024) 110598

Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet



Encrypted malicious traffic detection based on natural language processing and deep learning

Xiaodong Zang [a,b,c,*], Tongliang Wang [a], Xinchang Zhang [b], Jian Gong [c], Peng Gao [a], Guowei Zhang [a]

[a] School of Cyber Science and Engineering, Qufu Normal University, QuFu, China
[b] Qilu University of Technology (Shandong Academy of Sciences) Shandong Computer Science Center, Shandong Provincial Key Laboratory of Computer Networks, JiNan, China
[c] Key Laboratory of Computer Network and Information Integration, Ministry of Education, Southeast University, NanJing, China

ARTICLE INFO

Keywords:
Security and privacy
Anomaly detection
Artificial intelligence
Malicious traffic analysis
Encrypted traffic

ABSTRACT

The focus on privacy protection has brought much-encrypted network traffic. However, attackers always abuse traffic encryption to conceal malicious behaviors. Although researchers have proposed several enlightening detection methods, they must enhance the generalization ability or improve detection performance. Our inspiration is that the packet header fields, as do the underlying grammatical rules for constructing sentences, have a strict order. We consider the original packet as text and devise a robust approach with natural language processing and a deep learning model to improve the generalization ability and detection performance. We capture the critical keywords as characteristic representations of the traffic and design an adaptive domain generalization algorithm with a new loss function. It is robust against various datasets by generating more malicious samples to augment the minority of malicious samples. Simultaneously, we design an efficient feature selection algorithm, which obtains an optimal feature subset and reduces feature dimensions by 75.3%. To evaluate our work, we conducted extensive experiments with open-source datasets (CICIDS 2017, CICDDoS 2019, and USTC-TFC 2016), the synthetic dataset from IoT-23, and Internet backbone traffic (CERNET). Experimental results demonstrate that our proposal improves detection accuracy by up to 22.8% compared to others not using domain generalization algorithms and achieves an average detection latency of 0.67 s in the backbone. Besides, our work applies to the Industrial Internet of Things (IIoT) environment. It can be deployed at edge nodes to provide network security support for IIoT devices.

1. Introduction

Recently, traffic encryption technologies have been widely adopted to protect users' security and privacy when they surf the Internet. As reported by the Firefox website, encrypted web pages are increased by 80% from 2015 to 2021 [1–3]. In addition, according to statistics from Google, nearly 94% network traffic applies the transport layer security (TLS) protocol [4] for communication. As encrypted malicious traffic is highly concealed and high-risk, attackers always abuse traffic encryption to conceal their malicious behaviors, such as malware delivery, C&C channels, and data exfiltration. Therefore, identifying encrypted malicious traffic efficiently in the backbone is extremely important for network management and security monitoring.

Due to diverse traffic patterns and the emergence of various evasion technologies, detecting encrypted malicious traffic needs to be better addressed by traditional methods, e.g., port-based and payload-based approaches. In this context, it is imperative to devise efficient detection and defense strategies under encrypted traffic. Currently, there are three categories of encrypted malicious traffic detection approaches: (1) decryption followed by deep packet inspection (DPI)-based methods [5], (2) statistical analysis-based methods [6–8], and (3) machine learning-based methods [9–12]. Decryption followed by DPI is straightforward, and the detection results are promising. It conducts detection on the plaintext data after decrypting all encrypted network traffic. However, they cannot protect users' privacy and perform effective real-time network monitoring.

Statistical analysis-based methods use statistical flow features, such as average packet length and the number of incoming and outgoing bytes, to distinguish encrypted ones without decryption keys. However, these methods have a high false positive rate due to the need to

* Corresponding author at: School of Cyber Science and Engineering, Qufu Normal University, QuFu, China.
E-mail address: xdzang@qfnu.edu.cn (X. Zang).

https://doi.org/10.1016/j.comnet.2024.110598
Received 4 March 2024; Received in revised form 5 June 2024; Accepted 9 June 2024
Available online 11 June 2024

collect more long-time traffic flows to obtain higher quality characteristics [13]. Therefore, it is not suitable for deployment in realistic network environments online. Machine and deep learning, a branch of artificial intelligence, have been widely applied in network security monitoring. They collect enough benign and encrypted malicious traffic and automatically extract features from traffic metadata [12,14] for a prediction or classification task. Machine and deep learning-based methods have become mainstream in detecting encrypted malicious traffic.

Although machine and deep-learning-based detection schemes do not need to decrypt the encrypted traffic, they still face some challenges. Firstly, machine or deep learning-based detection schemes assume that the training and testing data are independent and identically distributed. In practice, we find that the training data always comes from the open-source benchmark datasets, while the testing is often the actual traffic, which violates the assumption. In our work, we take this phenomenon as lacking domain generalization ability. Besides, in the natural network environment, benign samples occur more frequently than malicious ones, which causes a data imbalance problem [15]. Secondly, most approaches rely on manually extracting statistical features. However, the features are designed for a specific scenario. The versatility and suitable feature selection mechanism need to be guaranteed.

This paper devised a novel framework with a natural language processing and deep learning model to solve the above problems. Detecting malicious traffic behaviors via packet analysis is feasible, as the multiple-step attack patterns are stealthy in the original packets. Our inspiration is that the packet header fields, as do the underlying grammatical rules for constructing sentences, have a strict order. Therefore, we capture the critical keywords in the packet's header and consider their field's value as a representation of traffic traits. We conduct an in-depth analysis of low detection performance in data processing. We over-sample the encrypted malicious traffic and design an enhanced robust algorithm to solve data imbalance distribution and achieve robust detection. We use the Albert model to convert qualitative data in traffic packets into quantitative data and extract 312 dimensions critical keywords with corresponding weights in the packet's header (e.g., TTL, Packet size). We devise an efficient feature selection algorithm to enhance its real-time and finally apply a 1D-CNN model to identify encrypted malicious traffic.

We prototype our system and deploy it in the wild. Before deploying it, we conducted extensive experiments with open-source datasets (CICIDS 2017, CICDoS 2019, USTC-TFC 2016) to evaluate its performance. Experimental results demonstrate that it can distinguish attacks under different datasets with an accuracy rate of 97.3%, which is much better than other similar alternatives. Besides, in the wild, we can detect encrypted malicious traffic not found by other works and achieve real-time detection with an average detection latency of 0.67 s. We also used a synthetic IoT dataset to verify that our work can apply to the IIoT. We compared it with similar alternatives and found our model performs better. The contributions are as follows.

(1) We propose a novel detection framework with natural language processing and a deep learning model under an encryption scenario. Our system can achieve real-time and robust detection in high-throughput networks.

(2) We conduct domain generalization analysis and devise an adaptive domain generalization algorithm with a new loss function. Our algorithm makes the data distribution even more diverse among different traffic categories. Compared with other works without domain generalization, our work improves the detection accuracy by at most 22.8%.

(3) We design an efficient additive increase-based random walk feature selection algorithm (AIRW). It can reduce the feature dimension by 75.3% and reduce detection time by 53.6%.

(4) We implement and deploy our prototype system in the China Education Research Network backbone (CERNET). We conducted an extensive evaluation, and with real-world deployment, we found some malicious cases that other detection approaches had not identified.

Organization: Section 2 surveys the related works of encrypted malicious traffic detection. Section 3 elaborates on the proposed approach, including data preprocessing, feature extraction, selection, and encrypted malicious traffic classification. Section 4 details the experiments with different datasets. Section 5 concludes the paper and gives our future work.

2. Related works and design goals

This section surveys the related works of encrypted malicious traffic detection. We broadly classify these techniques into decryption followed by DPI-based (DFDPI), statistical analysis-based (SA), and machine learning-based methods (ML). We survey them and introduce our design goals.

2.1. Related works

(1) **Decryption followed by DPI-based methods.** The deep packet inspection (DPI) technique requires the packet payload to be plaintext and constantly analyzes the available information in the application layer. However, with the increased encrypted traffic, this technique needs to decrypt all encrypted network traffic [16]. Therefore, researchers have developed other solutions using DPI techniques without decrypting the encrypted traffic. For example, [17] proposed a method using supervised learning to automate the rules for specific protocols in PGSM. Testing the performance of packets within these rules makes it possible to determine when and which signatures the target packets should match. [18]designed an OpenFlow-enabled DPI approach, which extracted the features of accessible payloads and linguistic features in the unencrypted packets, extracted the packets' notable features in the encrypted packets, and used a decision tree to detect malicious payloads.

(2) **Statistical analysis-based methods.** Techniques based on statistical analysis always use the statistical features of the flow. We divided these statistical features into two categories: protocol-agnostic numerical features and protocol-specific features [12,19–22]. The former includes packet-based and session-based characteristics. Packet-based characteristics, such as the average packet and the number of bytes in the packet, are always collected from the packet level. At the same time, the session-based features refer to the flow duration, total bytes, or total number of packets in each flow. The latter are encrypted protocol characteristics related to a specific protocol, e.g., handshake field features in TLS,[23,24]. [13] proposed a novel system called ME-Box to achieve reliable detection of malicious encrypted traffic. It first uses middleboxes to evaluate the trust degrees of encrypted traffic. When the encrypted flow is suspicious, the middleboxes request the session keys and provide evidence for the evaluation results. [6] developed a machine learning-based anomaly traffic detection framework (OADSD), which employs distributed dynamic feature extraction (DDFE) to extract representative features from raw traffic directly.

(3) **Machine learning-based approaches.** With the development of computer hardware, machine learning and deep learning as a branch of artificial intelligence have received much attention in security monitoring. [25,26] proposed real-time robust malicious traffic detection solutions with machine learning. The former could detect unknown encrypted malicious traffic with a clustering-based method by computing the interaction features of the flow. The proposed unsupervised graph learning technique can identify abnormal patterns by analyzing the connectivity and sparsity features of the graph. The latter performed frequency domain feature analysis, devised an automatic encoding vector selection algorithm to reduce the time-consummation of the manual parameter selection and accomplished real-time and robust detection. In addition to that, researchers have proposed many

Table 1
Summary of related works.

Category	OD	FS	Low Latency	Unknown Attacks	Robust Detection	Realtime Detection	Key technology
DFDPI	Packet	*N*	*N*	*N*	*N*	*N*	OpenFlow-enabled DPI [18]
	Packet	*N*	*N*	*N*	*P*	*N*	PGSM [17]
	biFlows	*P*	*N*	*P*	*N*	*P*	Heuristic statistical testing [19]
SA	Packet	*N*	*N*	*N*	*N*	*N*	Isolation Forest and K-means [20]
	Packet	*Y*	*P*	*P*	*N*	*N*	Density peaks clustering [23]
	Flows	*N*	*N*	*N*	*Y*	*N*	Evidence verification [13]
ML	Packet	*Y*	*N*	*P*	*P*	*N*	Feature transformation [38]
	Flows	*N*	*N*	*P*	*Y*	*Y*	Machine learning [26]
	Packet	*N*	*P*	*P*	*N*	*P*	Deep learning [39]
	Flows	*N*	*N*	*P*	*Y*	*Y*	Adaptive Random Forests [27]
	Packet	*P*	*P*	*P*	*N*	*N*	Machine learning and NLP [11]
	Packet	*N*	*N*	*P*	*P*	*N*	Deep learning [28]
	IDSlogs	*N*	*N*	*P*	*P*	*P*	Ensemble-based deep learning [31]
Our work	Packet	*Y*	*Y*	*Y*	*Y*	*Y*	NLP and Deep learning

solutions with deep learning [7,9,11,27–33]. [28] designed a multi-level feature fusion model. The model can automatically extract byte, timing, and other statistical features. They also devised an adaptive balanced training method to improve the data distribution and detection performance. [11] introduced a natural language processing method (TF-IDF model) to capture the traffic feature and used machine learning (Gradient boosting, random forest, AdaBoost classifier) and deep learning techniques (CNN neural network)to distinguish malicious traffic. A large number of wireless devices connect to the network, making IoT security a focal point for intrusion detection [34]. There are various intrusion detection systems in IoT, such as hybrid, anomaly-based, signature-based, and specification-based [35]. [36] uses a blockchain-based Radial Basis Function Neural Network to improve data integrity and intelligent decision-making in various Internet of Drones environments. [37] employs Markov Decision Processes and Deep Learning to address IoT task latency issues.

We provide a detailed comparison of encrypted traffic detection in Table 1, where OD represents original data, and FS is short for feature selection. We use Y to indicate that the corresponding column meets the conditions, N is unsatisfied, and P is partially satisfied. The above surveys show that machine and deep learning-based solutions have achieved satisfactory detection tasks within the given dataset.

2.2. Design goals

Our work is to devise a robust encrypted malicious traffic detection system to achieve efficient detection in high throughput networks. Notably, our system achieves the following two goals, which need to be better addressed in the previous research.

(1) Robust accurate detection. Machine learning-based approaches assume that the dataset is independent and identically distributed. However, in the wild, it often violates the assumption. Our system should be able to detect encrypted malicious traffic under various datasets and can capture evasion attacks, such as the attackers injecting noise packets in benign applications.

(2) Low detection latency in the backbone. Our detection system should apply in high throughput networks, e.g., backbone, to conduct high-speed traffic analysis and real-time detection. To this end, we use a natural language processing model to extract useful information and devise a lightweight feature selection approach, which reduces the feature dimension to 75.3%.

3. Encrypted malicious traffic modeling approach

Our detection system includes four modules: data processing, feature extraction, feature selection, and encrypted malicious traffic classification, as shown in Fig. 1. We pre-process the original imbalanced encrypted traffic packet with the enhanced domain generalization algorithm. In the feature extraction and selection stage, we consider all packet headers as text, use the ALBERT model to convert the qualitative data into quantitative data, and capture the best-weighted feature subset. Finally, we use a 1D-CNN model to identify encrypted malicious traffic.

3.1. Data processing

Data Cleaning: The experimental dataset originates from a real-world environment. It includes packets irrelevant to encrypted malicious traffic detection (e.g., plaintext data on ports 80 or 21). To enhance the relevance and purity of the dataset, we discarded these irrelevant packets using Wireshark filters. First, we parsed the pcap files using tools and libraries such as Scapy and Pyshark to discard plaintext network traffic. To ensure data quality and avoid interference with model training, we applied the dropna and duplicated functions from the Pandas library to remove incomplete, duplicate, and retransmitted packets [19]. For the known encrypted traffic (e.g., encrypted data on ports 443 or 500), we only extracted the packet header of encrypted traffic without decrypting the packet's load content. We use the randomness-testing library for other unknown encrypted traffic to perform randomness detection on their packet load, assessing the randomness and diversity of the packet load [40]. We filter out the non-encrypted packets to ensure our final dataset contains only encrypted ones. As we aim to find the malicious behavior in the encrypted traffic, we do not filter the DNS traffic in our dataset. Next, we transformed the extracted packet heads into a text format that the model can process.

Robust Generalization Training: After obtaining the formatted encrypted text data, we find that benign traffic occurs more frequently than malicious traffic, which causes data imbalance. Data imbalance will cause unsatisfactory detection performance as the conventional approaches always assign equal importance to the class samples. Additionally, different datasets may exhibit varying data distributions. The training data comes from open-source benchmark datasets. In contrast, the testing data is often the actual traffic. Therefore, assuming that the data are independent and identically distributed can lead to overfitting and poor generalization.

However, many data augmentation methods (such as SMOTE and ROS) have disadvantages. SMOTE may generate unrealistic samples at class boundaries, affecting classification performance. ROS may introduce redundant data, increasing the risk of overfitting. Therefore, we devise an adaptive domain generalization algorithm based on the variational autoencoder with a new loss function to tackle the above issues.

Variational autoencoder is one of the methods in disentanglement representation learning (DRL) [41], which is widely applied in text generation, style conversion, and semantic understanding. We use VAE's encoder to convert the textual features into potential representations of the traffic to make the generated text data more diverse and fill in the differences between different data distributions. We analyze the

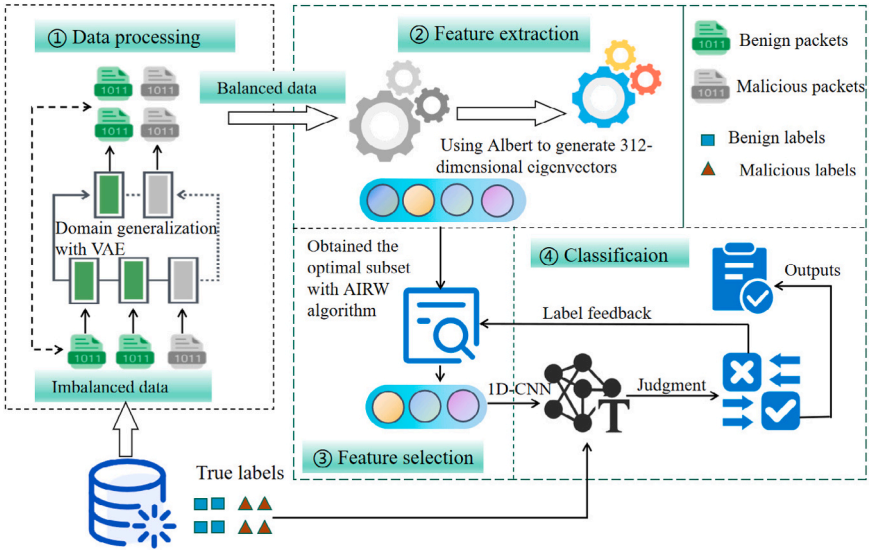


Fig. 1. Methodology of Flow Diagram.

gradients contributed by each feature of the IP packets. Intuitively, given the trained VAE and a list of IP packets, if the reconstruction error changes quite a lot when a small amount varies a particular feature of the IP packet, this feature at its current value is anomalous.

As our method involves augmenting minority malicious samples, bias may be introduced during the data augmentation. The model may overfit these samples with the increased proportion of augmented malicious samples in the training data. In our work, we use Dropout during training to prevent the model's overfitting. Considering that if the generated augmented samples are not realistic enough, the model may learn incorrect features. We introduce adversarial losses to evaluate the authenticity of the generated text and prevent noise interference in the generation process. The introduced adversarial losses function avoids bias in the generated samples. Experimental results demonstrate that it solves the problem of overfitting caused by oversampling and poor generalization ability in different datasets.

Algorithm 1 The pseudo-code of an enhanced domain generalization algorithm.

Input:
A list of oversampled encrypted traffic packets:$p' = \{p'_1, p'_2, p'_3 ... p'_m\}$
Output:
New encrypted traffic data: $q = \{q_1, q_2, q_3 ... q_m\}$
function EnhancedGomainGeneralizationFunc(p', q)
  p'_i= real_data(p') / * Read real samples* /
  $\mu, \log(\sigma^2)$ = Encoder(p'_i) / * Obtaining the mean and standard deviation* /
  $z' = \mu + \epsilon \cdot \exp(\log(\sigma^2))$ / * Obtaining the temporary variable z', where $\epsilon \sim N(0,1)$. * /
  $\hat{p}'_i$= Decoder(z') / * Generating reconstructed samples $\hat{p}'_i$ from z' using Decoder()* /
  $\hat{p}'_{\text{gen}}$ = Decoder (z_{gen})
  Calculating the loss function $\mathcal{L}_{\text{adv}}$ with $D(p')$ and $D(\hat{p}'_{gen})$
  $\mathcal{L} = \mathcal{L}_{\text{rec}} + \lambda_{\text{kl}} \mathcal{L}_{\text{kl}} + \lambda_{\text{adv}} \mathcal{L}_{\text{adv}}$ / * Calculating the overall loss function * /
  update_parameters($\mathcal{L}$) / * Update network parameters to minimize $\mathcal{L}$ * /
  Using the generated training model to obtain the output data q
return q

The new loss function addresses overfitting due to oversampling and poor generalization across different datasets. It combines reconstruction loss ($\mathcal{L}_{rec}$), Kullback–Leibler divergence loss ($\mathcal{L}_{\text{kl}}$), and adversarial loss ($\mathcal{L}_{adv}$), as shown in Eq. (1).

$$\mathcal{L} = \mathcal{L}_{\text{rec}} + \lambda_{\text{kl}} \mathcal{L}_{\text{kl}} + \lambda_{\text{adv}} \mathcal{L}_{\text{adv}} \tag{1}$$

We use the p' to denote oversampled encrypted traffic packets, which are encoded into a temporary variable z' by using a probabilistic encoder $q_\phi(\boldsymbol{z}' \mid \boldsymbol{p}')$. We use a probabilistic decoder $p_\theta(\boldsymbol{p}' \mid \boldsymbol{z}')$ to reconstruct the original sample from z'. $\mathcal{L}_{\text{rec}}$ computes the difference between the sample reconstructed $\hat{p}'_i$ from the latent representation z' and the original input p', as shown in Eq. (2). The more minor the loss ($\mathcal{L}_{rec}$), the closer the generated sample is to the original sample.

$$\mathcal{L}_{\text{rec}} = -\mathbb{E}q_\phi(\boldsymbol{z}' \mid \boldsymbol{p}') \left[\log p_\theta(\boldsymbol{p}' \mid \boldsymbol{z}')\right] \tag{2}$$

Kullback–Leibler divergence loss ($\mathcal{L}_{\text{kl}}$)measures the difference between the temporary variable distribution $q_\phi(\boldsymbol{z}' \mid \boldsymbol{p}')$ and the prior distribution$p_\theta(\boldsymbol{p}' \mid \boldsymbol{z}')$. λ_{kl} controls the weight of the KL loss function during training, where

$$\mathcal{L}_{\text{kl}} = \mathcal{D}\text{kl}\left(q_\phi(\boldsymbol{z}' \mid \boldsymbol{p}') | p(\boldsymbol{z})\right) \tag{3}$$

We introduce adversarial loss ($\mathcal{L}_{adv}$) to evaluate the authenticity of generated samples by adding a discriminator network $D(\boldsymbol{p}')$. λ_{adv} is a weight of adversarial loss, where, $\boldsymbol{p}' \sim p$ data $(\boldsymbol{p}')$ denotes the sampled real sample p', $\hat{\boldsymbol{p}}'_{gen} \sim p$ data $(\hat{\boldsymbol{p}}'_{gen})$ denotes the sampled reconstructed sample $\hat{\boldsymbol{p}}'_{gen}$ from the generated model, $D(p')$ and $D(\hat{\boldsymbol{p}}'_{gen})$ are the discriminator's probability of classifying real samples p' or the reconstructed samples $\hat{\boldsymbol{p}}'_{gen}$ as ture or false, respectively. The generative model improves the realism of the generated samples by minimizing the adversarial loss and, in the meantime, ensures that the objective function of the variational autoencoder is unchanged. Algorithm 1 gives its pseudo-code, which aims to minimize the loss function.

$$\mathcal{L}_{\text{adv}} = \mathbb{E}_{\boldsymbol{p}' \sim p \text{ data } (\boldsymbol{p}')}[\log D(\boldsymbol{p}')] + \mathbb{E}_{\hat{\boldsymbol{p}}'_{gen} \sim p \text{ data } (\hat{\boldsymbol{p}}'_{gen})}\left[\log\left(1 - D\left(\hat{\boldsymbol{p}}'_{gen}\right)\right)\right] \tag{4}$$

3.2. Feature extraction

The packet header fields, as do the underlying grammatical rules for constructing sentences, have a strict order. In this section, we consider all packet headers as text and use a natural language processing model (ALBERT) to convert the qualitative data in traffic packets into quantitative data.

ALBERT uses a transformer encoder as its core component. The transformer encoder comprises multiple layers with the same structure.

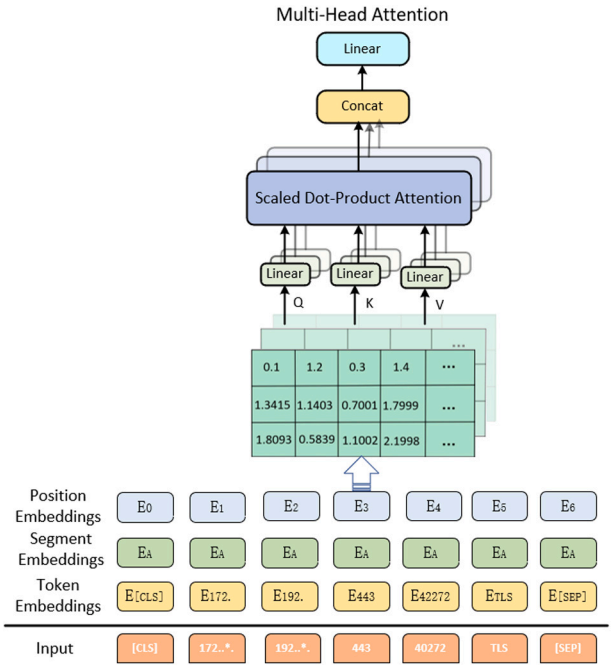


Fig. 2. Albert feature extraction process.

Each contains a self-attention mechanism and a feed-forward neural network [42–44]. The attention mechanism captures the dependencies between keywords in the sequence. The feed-forward neural network further extracts features and performs non-linear transformations. The training process and gradient propagation are stabilized by applying residual connections and layer normalization. Finally, high-level features are extracted and integrated to generate the final feature representations. Fig. 2 shows the framework of Albert.

In the work, we represent each input as a list of vectors consisting of unique tokens (e.g., [CLS], [SEP]) and segment IDs. We first use ALBERT to tokenize the text and convert each word into its corresponding ID. We then convert the tokenized text into the format that ALBERT can process. Additionally, position encoding is generated for each word segment. Thirdly, we use the embedding layer to convert the input word IDs into corresponding word vectors while adding position encodings. Finally, we use the transformer encoder for better feature representation. Algorithm 2 shows the pseudo-code of the feature extraction process.

We use a five-tuple (e.g., 172.16.0.1, 192.168.10.51, 443, 40272, TLS) as an example to explain how to convert qualitative data into quantitative data. (1) Using the tokenizer to convert the five-tuple string into a token sequence (e.g., ['172', '.', '16', '.', '0', '.', '1', '192', '.', '168', '.', '10', '.', '51', '443', '40272', 'TLS']), and then converting the token sequence into the corresponding ID sequence (e.g., [101, 10, 102, 10, 103, 10, 101, 101, 10, 102, 10, 103, 10, 104, 105, 106, 107]). (2) Converting the ID sequence into a tensor format suitable for PyTorch or TensorFlow and adding unique tokens [CLS] and [SEP] at the beginning and end of the ID sequence (e.g., [CLS, 101, 10, 102, 10, 103, 10, 101, 101, 10, 102, 10, 103, 10, 104, 105, 106, 107, SEP]). (3) Generating the word embedding matrix, which has a size of (vocabulary size, embedding dimension). (4)Generating position embedding vectors, which provide positional information for each token in the sequence. As shown in Eqs. (5) and (6), d_{model} represents the dimension of the embedding vectors. For even-dimensional indices (2i), the $sin()$ computed the values of the position encodings, while for odd-dimensional indices (2i+1), the $cos()$ computed the values. Since our input is a single text paragraph, we set segment embeddings to zeros. Finally, the word embedding vectors are added to the corresponding position encoding vectors to form the input embedding vector sequence.

$$P_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \tag{5}$$

$$P_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right) \tag{6}$$

After that, the Albert model extracts high-level features of the input sequence by stacking multiple layers of self-attention mechanisms and feed-forward neural networks. The self-attention mechanism allows each word to attend to information from other words in the input sequence. In contrast, the multi-head mechanism enables the model to extract features from different subspaces. The input sequence is into query (Q), key (K), and value (V) vector spaces, as shown in Eq. (7). The attention scores are obtained by dividing the dot product of the query vector and key vector by a scaling factor ($\sqrt{d_k}$) and then converting them into attention weights using the softmax function. d_k represents the feature dimension within the attention head. This process repeats using different weight matrices multiple times, resulting in multiple self-attention outputs. A linear layer transforms these outputs to obtain the final multi-head self-attention output. The output of the multi-head self-attention mechanism is fed into a feed-forward neural network for further processing, as shown in Eq. (8). The final output is a sequence in which each position corresponds to a feature in the input sequence and contains contextual information about that feature.

$$Self - attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \tag{7}$$

$$\text{FFN}(x) = \text{ReLU}(W_1 \cdot x + b_1) \cdot W_2 + b_2 \tag{8}$$

In Eq. (8), x is the output of the multi-head self-attention mechanism after layer normalization. W_1 and W_2 are weight matrices. The former maps the input vector x to a higher-dimensional intermediate representation, while the latter maps the non-linearly transformed intermediate representation back to the original dimension. b_1 and b_2 are the corresponding bias vectors. ReLU is a non-linear activation function that applies a non-linear transformation to the result of the linear transformation, enabling the model to capture complex patterns and features.

Algorithm 2 Eigenvector extraction based on Albert.

Input:
Text datasets: $text = \{q_1, q_2, q_3 ... q_m\}$
Output:
Albert's eigenvector: $encoder\_output = \{v_1, v_2, v_3 ... v_m\}$

```
/ * Tokenization * /
tokens = tokenizer.tokenize(text)
/ * Convert tokens to IDs, including segment encoding and position encoding * /
input_ids, segment_ids, position_ids = (tokenizer.convert_tokens_to_ids(tokens),
    generate_segment_encoding(tokens),
generate_position_encoding(tokens))
/ * Embedding layer * /
embeddings = embedding_layer(input_ids, segment_ids, position_ids)
/ * Transformer encoder * /
encoder_output = transformer_encoder(embeddings)
return encoder_output
```

3.3. Feature selection

After feature extraction, we obtained 312-dimensions feature vectors. However, not all dimensions positively impact the detection accuracy. Therefore, feature selection is essential to obtain high-quality

features that directly affect detection accuracy. Although filtering-based and wrapper-based methods are widely used, they either fail to capture the best feature subset or take a long time for model training [45].

We devise an AIRW (additive increase-based random walk feature selection algorithm) to capture the optimal feature subset. The feature selection method contains two steps. It first ensures a locally optimal space in our initial eigenvector (312 dimensions). Hence, we gradually increase exponentially to choose the subset from 16 to 256 dimensions, e.g., 16, 32, 64, 128, and 256. We obtained the optimal feature range of [64, 128] in the first step. After identifying the local optimal space, we further employ a random walk strategy to optimize the feature dimension within this local space. The specific steps are as follows. (1) Initialization. Setting the current dimension to 64 and record its performance. (2) Random Perturbation. Small random perturbations are made to the current dimension, such as increasing or decreasing by a few dimensions. (3) Performance Evaluation. Evaluating the performance of the feature subset at the new dimension. (4) Select the Optimal Dimension. If the performance at the new dimension is better than the current dimension, update the current dimension to the new dimension and record the new performance. (5) Repeat the Process. Repeating steps 2 to 4 until the performance no longer significantly improves or a preset number of iterations is reached. The pseudo-code of it is shown in algorithm 3.

Algorithm 3 The pseudo-code of additive increase-based random walk feature selection algorithm.

Input:
Albert's eigenvector:$encoder\_output = \{v_1, v_2, v_3 ... v_{312}\}$
Output:
Eigenvectors of optimal subsets: $best\_eigenvector = \{v_1, v_2, v_3 ... v_m\}$
for each $i \in \{16, 32, 64, 128, 256\}$ **do**
  $result.append(M(v_i))$
end for
$max\_value = \max(result)$ / * Selecting the optimal dimension * /
$max\_index = \arg\max(result)$ / * The index of the optimal dimension * /
if $(M(\text{max\_index} - 1) - M(\text{max\_index} + 1) > 0)$ **then**
  [max_index × 2^{-1}, max_index] / * Selecting the optimal feature range * /
else
  [max_index, max_index × 2^{+1}]
end if
Setting current_index as random[max_index, max_index × 2^{+1}]
for each iteration **do**
  Generating a random number(*num*) between -1 and 1
  if *num* >= 0 **then**
    current_index=current_index-1
  else
    current_index=current_index+1
  end if
  if (current_index < start_index) or (current_index > end_index) **then**
    break
  end if
  $best\_result.append(M(v_{current\_index}))$
end for
best_eigenvector = arg max(best_result)
return *best_eigenvector*

The deep learning models work like a black box, making it difficult to understand their decision-making process. To improve the interpretability of deep learning models and understand the reasoning behind anomaly detection, we utilize the SHAP (Shapley Additive explanations) algorithm to measure the importance of each feature [46]. The SHAP algorithm is based on the concept of Shapley values from cooperative game theory. It computes the contribution of each feature

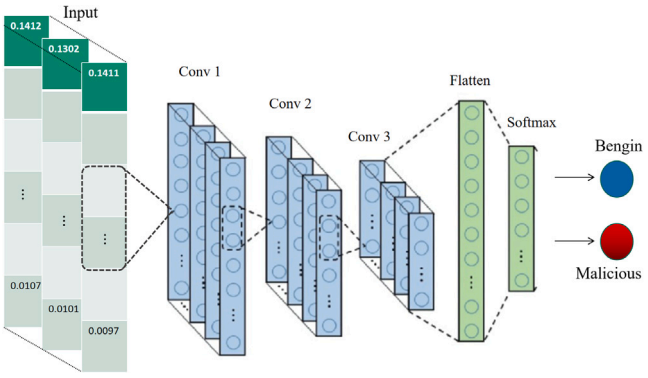


Fig. 3. 1D-CNN model.

in the model to explain which features contribute the most to the detection result, providing an intuitive explanation of the model's decisions. For a given feature j, sample i, the SHAP value ϕ_i^j is calculated as follows:

$$\phi_i^j = \sum_{S \subseteq \{1,2,\ldots,p\} \setminus \{j\}} \frac{|S|!(p - |S| - 1)!}{p!} [f(x_{iS \cup j}) - f(x_{iS})] \tag{9}$$

Where p represents the total number of features. $|S|$ denotes the size of feature set(S). $f(x_{iS})$ represents the model's predicted output for sample i under S. $f(x_{iS \cup j})$ represents the model's predicted output for sample i after adding the value of j to S. We randomly select 25 features from the first 77 dimensions and another 25 features from the remaining dimensions. With the SHAP algorithm, we find that the features from the first part significantly impact the classification task (e.g., Protocol Type, Packet Length, Payload Length, Packet Arrival Time, etc.). The features are also found in previous references of manual feature extraction [2,9,12]. However, the features from the latter part have a minimal impact on the classification. It is proved that our feature selection method is meaningful.

3.4. Classification

CNN has been frequently employed in two-dimensional signal processing (e.g., image identification) [47,48]. Recently, it has shown outstanding performance in encrypted traffic analysis. Considering the feature vector obtained above and the one-dimensional CNN (1D-CNN) application in detecting network attacks (e.g., it can capture the spatial dependencies information between different bytes of packets and offer more helpful information to identify and classify encrypted malicious traffic) [49], we convince that 1D-CNN is an optimal classier, and the experimental results confirm our claim. We briefly outline the 1D-CNN model, which adopts four layers: convolution, pooling, flatten, and a fully connected layer, as shown in Fig. 3.

In 1D-CNN, the convolutional layer captures local information and perceives it through a sliding window to obtain locally relevant information. The convolutional layer uses shared weights on the convolution kernel, significantly reducing the number of parameters in the network and the risk of overfitting. Assuming our optimal feature subset is X, the convolution kernel is W, the bias is b, and the stride is s. After adding the ReLU activation function, the output feature calculation formula is shown in Eq. (10).

$$Y(i,j) = \text{ReLU}\left(\sum_{m=0}^{M-1}\sum_{n=0}^{N-1} X(i \cdot s + m, j \cdot s + n) \cdot W(m,n) + b\right) \tag{10}$$

In 1D-CNNs, the convolutional layer is usually followed by a pooling layer, a flatten layer, and a fully connected layer. We adopt the mean pooling method in the pooling layer. Average pooling can reduce the feature's mapped dimension and the model's computational complexity

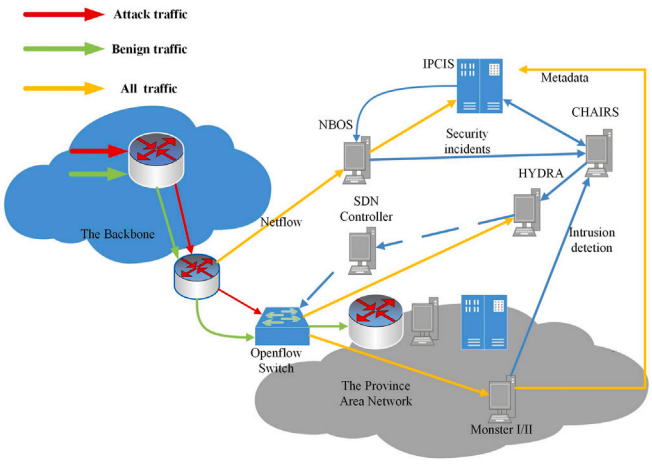


Fig. 4. The overall architecture of CERNET.

and number of parameters. It can prevent overfitting and improve the model's generalization ability. The average pooling method is shown in Eq. (11).

$$Z_{i,j} = \frac{1}{k' \times s'} \sum_{m=0}^{k'-1} \sum_{n=0}^{s'-1} Y_{i,j \times s' + n} \tag{11}$$

where k' denotes the size of the pooling window, s' represents the pooling stride, i represents the row index of the output tensor, and j represents the column index.

The flatten layer does not introduce any new parameters and weights. It simply rearranges the data without modifying the content. We use it to convert a multidimensional input X into one-dimensional. For example, for an input X of dimension (N, C, L), the flatten operation converts it to a one-dimensional one (N, C*L). We use the fully connected layer to learn the non-linear dependencies of the data. It connects all the neurons in the previous layer, and each connection has a weight. Since our encrypted malicious detection is a binary classification task, the calculation method after adding the Sigmoid activation function is shown in Eq. (12).

$$Y_{fc} = \text{Sigmoid}(Z_{flat} \cdot W_{fc} + b_{fc}) \tag{12}$$

Where W_{fc} is the weight matrix, Z_{flat} is the input feature vector after performing matrix multiplication in the flattened layer, and b_{fc} denotes the bias vector. With 1D-CNN, we classify the malicious network packets using the packet-level features extracted from the Albert model.

4. Evaluation

We use open-source datasets and Internet traffic (CERNET) to verify the scalability in different network environments. The experiment preparation includes a description of the dataset and evaluation metrics, experimental results analysis, and a comparison of our work with other state-of-the-art methods on malicious traffic detection tasks. We performed all the experiments on a 2-way Intel Xeon server with one Intel(R) Xeon(R) CPU E5-2650 processor that contains eight cores at a frequency of 2.00 GHz and 128 GB of memory. Tesla P40 is also used to accelerate the training and test phase. Experimental results demonstrate that our work outperforms other similar alternatives.

4.1. Datasets and evaluation metrics

CICIDS 2017. The University of New Brunswick published the public datasets CICIDS 2017 and CICDDoS 2019 for downstream tasks of encrypted traffic classification. Researchers captured the traffic in a controlled network environment for five consecutive days. They subdivided traffic attack scenarios into Brute Force FTP, Brute Force SSH, DoS, Heartbleed, Web Attack, Infiltration, Botnet, and DDoS. Therefore, the attack types are abundant. We randomly selected 2.38 GB of traffic for the experiment, as shown in Table 2.

Table 2
The details of the open-source datasets.

Open-source datasets	Category	Types subdivided	Proportion
CIC-IDS 2017	Benign traffic	Benign traffic	54%
	Brute Force	FTP-Patator	2.9%
		SSH-Patato	2.4%
	DoS/DDoS	DoS slowloris	5.4%
		DoS Hulk	5.4%
	Web Attack	Web Attack C XSS	8.1%
		Web Attack C Sql	8.1%
	Botnet	Botnet ARES	10.8%
	Port Scan	Port Scan	2.9%
USTC-TFC 2016	Benign traffic	Benign traffic	52.8%
		Cridex	11.1%
	Malicious traffic	Htbot	6.2%
		Miuref	5.9%
		Nsis-ay	6.4%
		Shifu	5.7%
		Virut	5.5%
		Zeus	6.4%
CIC-DDoS 2019	Benign traffic	Benign traffic	55.5%
	TCP based attacks	MSSQL	11.1%
		SSDP	5.5%
	TCP/UDP based attacks	DNS	3.7%
		LDAP	7.4%
		SNMP	5.5%
	UDP based attacks	NTP	3.9%
		TFTP	7.4%
IoT-23	Benign traffic	Benign traffic	53.4%
	C&C	C&C-FileDownload	3.9%
		C&C-HeartBeat	5.5%
		C&C-Torii	4.6%
	Okiru	Okiru-Attack	9.8%
	DDoS	DDoS	10.2%
	Port Scan	Port Scan	12.6%
The real traffic	Benign traffic	Benign traffic	50.8%
	Malware	C&C	10.4%
	Web Attack	Https,SQL,TLS	9.8%
	Brute Force	Scan	8.9%
	Flooding Attack	TCP,UDP,DNS,NTP	8.4%
	Alerts	Alerts,Logs	11.7%

CICDDoS 2019. The CIC-DDoS 2019 dataset contains benign and the most up-to-date common DDoS attacks. In addition to that, it also includes the detection results of CICFlowMeter-V3, which is a network traffic tool based on labeled flows. This dataset has modern reflective DDoS attacks like PortMap, NetBIOS, LDAP, MSSQL, UDP, UDP-Lag, SYN, NTP, DNS, and SNMP. We randomly collected 1.78 GB of traffic for the experiment.

USTC-TFC 2016. The USTC-TFC 2016 dataset is released by the Cybersecurity Lab at the University of Science and Technology of China [50]. The dataset consists of two parts. The first part includes malicious malware traffic (e.g., Htbot, Nsis-ay, Zeus) from 10 types of public websites collected by CTU researchers from 2011 to 2015 in real network environments. The second part contains ten types of benign traffic (e.g., Gmail, Bit Torrent, Bit Torrent, Ftp) collected using IXIA BPS, a professional network traffic simulation device. The USTC-TFC 2016 dataset is 3.71 GB, and we randomly collected 2.15 GB for the experiment.

The real traffic. Our real traffic was collected at CERNET from 6/1/2023 to 6/7/2023 [51]. The topology of CERNET is shown in Fig. 4. Our original packets are collected at the Network Behavior Observation System (NBOS). NBOS monitors and manages the service quality and security status of CERNET by analyzing the incoming data streams. When it finds abnormal events, it sends alerts to CHAIRS.

Table 3
Model parameter settings.

Model	Parameter	Value
ALBERT	maximum_seq_length	512
	max_predictions_per_seq	20
	eval_batch_size	32
	learning_rate	0.0001
CNN	layers	3
	kernel_size	2
	learning_rate	0.0001
	dropout_rate	0.15

CHAIRS fuses heterogeneous threat intelligence (e.g., suspicious IP flows, malicious domain names, Alerts, Audit logs, suspicious C&C traces) to mine the multi-step attacks and track high-risk attack paths. Monster I/II is an intrusion detection system that analyzes the NetFlow and identifies the suspicious ones. HYDRA system receives network security events from the CHAIRS system. It converts the response rules into flow entries in the OpenFlow switch and blocks the attack traffic at the network boundary. We have collected 1.43 TB packets in a week. After processing the original traffic, we obtained 41.82 GB packets for the experiment.

Synthetic dataset. We randomly selected 2.67 GB from the IoT-23 dataset and 4.25 GB from the real traffic. The Stratosphere Laboratory, CTU University, Czech Republic, built the IoT-23 dataset. It contains communication traffic from twenty-three IoT devices. Twenty devices capture malware traffic, and three devices capture benign IoT traffic. We synthesized them to simulate network traffic in a real IIoT environment and used them to validate the capabilities for identifying malicious traffic in IoT devices. The volume of our traffic is 4.92 GB.

In the experiment, we use VirusTotal [52] to cross-validate the detection results of NBOS to construct the ground truth data for training the model. We upload the detection result to the VirusTotal website and collect the traffic that is regarded as malicious. We utilize $Reacll$, $OverallAccuracy$, $Precision$, and $F1-Score$ as the algorithm's evaluation criteria. $Recall$ indicates the ratio of samples classified as malicious among those malicious samples. $Overallaccuracy$ indicates the ratio of correctly identified samples to the total number of samples. $Precision$ indicates the ratio of malicious samples among those classified as malicious. The $F1-Score$ comprehensively evaluates the model's performance in detecting malicious samples. BR_i assesses the model's ability to balance risk when handling different types of samples, ensuring the model's stability and robustness. In Eq. (13) to 16, we use TP, FP, TN, and FN to denote the true positive, false positive, true negative, and false negative, respectively.

$$Reacll = \frac{TP}{TP+FN} \tag{13}$$

$$OverallAccuracy = \frac{TP+TN}{TP+TN+FP+FN} \tag{14}$$

$$Precision = \frac{TP}{TP+FP} \tag{15}$$

$$F1-Score = \frac{2*Precision*Recall}{Precision+Recall} \tag{16}$$

$$BR_i = \frac{attack\ samples\ of\ class\ i}{benign\ samples} \tag{17}$$

4.2. Experimental results analysis

In this section, we conduct a series of experiment validations in different classification algorithms evaluation, feature subset analysis, robustness analysis, and system performance analysis. Considering the ALBERT and 1D-CNN model parameters, we specified the maximum input length as 512 tokens to control the input data size and align with model requirements. We simultaneously set the batch size (32) to process 32 samples during the feature extraction. In the 1D-CNN, the Adam optimizer and an adaptive learning rate optimization algorithm adjust learning rates to accommodate parameter changes dynamically. We set a small learning rate (0.0001) to ensure stable convergence during training. Besides, we randomly discard a portion of neurons by setting the dropout rate to 0.15 to avoid overfitting. We show the model parameters in Table 3.

4.2.1. Different classification algorithms evaluation

We choose four classical classifiers to identify malicious traffic. They are SVM, random forest (RF), CNN, and Recurrent Neural Network (RNN). We analyze different kernel functions in the SVM classifier, e.g., linear, sigmoid, Polynomial, and radial basis functions (RBF), as shown in Fig. 5(a). In the RF classifier, we evaluate the impact with different numbers of decision trees, as shown in Fig. 5(b). The accuracy increases are not significant with the increase of decision trees. We select 60 decision trees for the following evaluation to avoid unnecessary computation. We then evaluate CNN and RNN classifiers with different convolution layers and hidden layer sizes, as shown in Figs. 5(c) and 5(d). Finally, we use the four metrics to evaluate the four classifiers. Fig. 6 shows that our 1D-CNN model has the highest accuracy .

4.2.2. Feature subset analysis

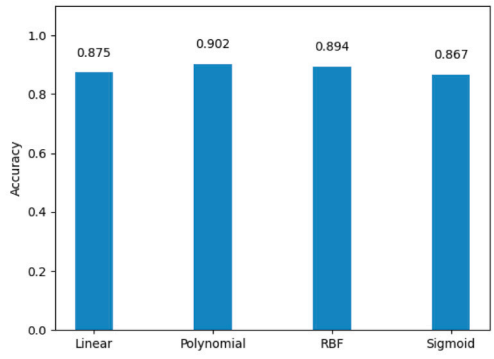
To ensure comprehensive feature learning, we conduct 56 iterations, and the model's weights are updated at each iteration. Finally, we have obtained the optimal feature subset (77 dimensions) with the devised feature selection algorithm. We analyzed the detection accuracy with different feature dimensions, as shown in Fig. 7. It demonstrates that the model's performance changes with varying feature dimensions. However, it achieves the highest level of performance when choosing 77 dimensions, and any over or under-specific feature dimensions will cause lower detection performance.

In addition to detection accuracy, we analyze the average training time of the CICIDS 2017, CICDDoS 2019, and USTC-TFC 2016 with complete feature combination and feature selection algorithm, as shown in Fig. 8. The factors affecting training time are determined by the computing power of different platforms and the feature dimensionality in the dataset. On the one hand, platforms with stronger computing power use less average training time, which is an objective factor. On the other hand, our additive increase-based random walk feature selection algorithm reduces the feature dimensions by 75.3%, which is the subjective factor in reducing training time. Although data distribution needs to be balanced during actual deployment, the robust algorithm requires very little time to process the oversampled malicious traffic.

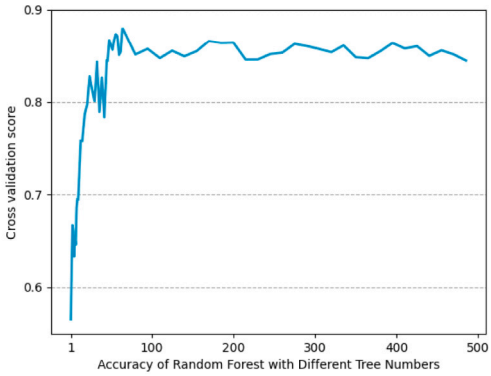
4.2.3. Robustness analysis

In this part, we utilize one of the datasets (CICIDS 2017) to evaluate the impact of λ_{kl} and λ_{adv} on the robustness. λ_{kl} controls the weight of the KL divergence loss function during training, and $\lambda_{kl} \in [0,1]$. λ_{adv} is a weight hyperparameter of adversarial loss, and $\lambda_{kl} \in [0,1]$. We introduce adversarial loss to improve the authenticity of the generated samples. The smaller λ_{kl}, the smaller the reconstruction error, and the better to reconstruct the input data, demonstrating the decoder's greater accuracy in restoring the original data. However, a more considerable λ_{adv} can enhance the impact of adversarial training and emphasize the authenticity of generated samples. We analyze the value of λ_{kl} and $\lambda_{adv}$to make the generated data more authentic and more robust. After continuous experiments, when λ_{kl} equals 0.032 and λ_{adv} equals 0.95, the CICIDS 2017 dataset becomes more balanced (RF = 1:1), and the algorithm is more robust. The details are shown in Fig. 9.

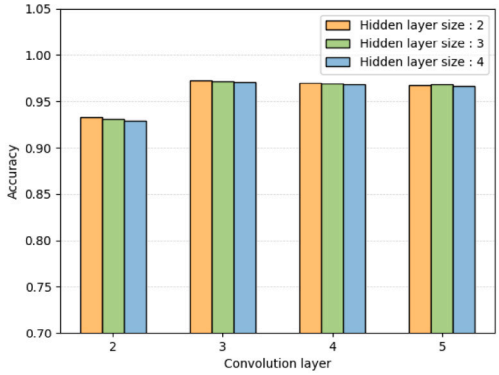
After obtaining reasonable parameters, we verify the robustness of our model with different datasets. Table 4 shows that when the model chooses the same trained and test data in CICIDS 2017, CICDDoS 2019,



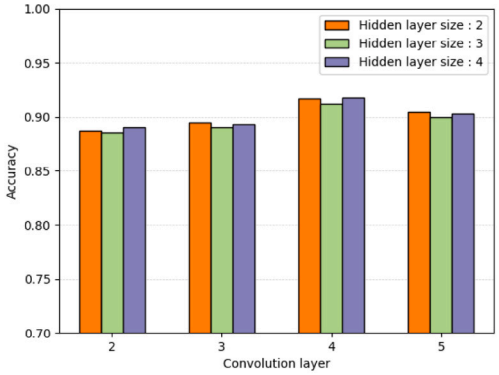
(a) Accuracy of SVM with different kernels



(b) Accuracy of RF with different tree numbers



(c) Accuracy of CNN with different convolution layer



(d) Accuracy of RNN with different convolution layer

Fig. 5. Accuracy of different models.

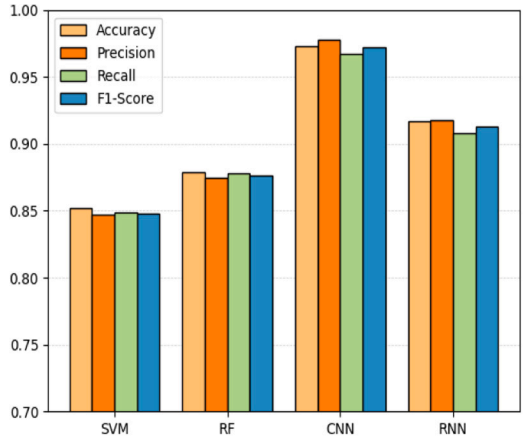


Fig. 6. Accuracy of different classification algorithms.

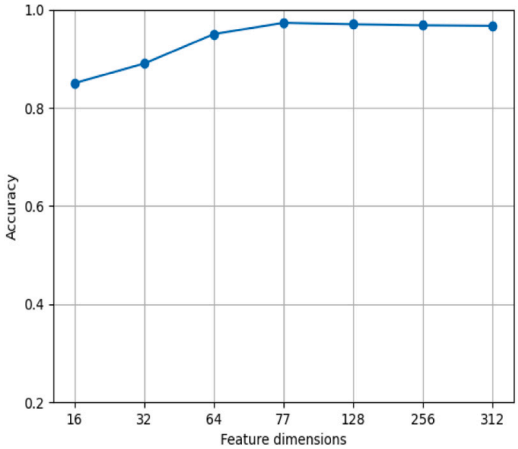


Fig. 7. Accuracy of different feature dimension.

or USTC-TFC 2016, nearly all the evaluation metrics can reach 99%. However, the detection performance could be better when the model selects different trained and test data with the devised robust algorithm. When we use algorithm 1 to optimize the dataset, the evaluation metrics can reach nearly 97%, demonstrating that our work has good generalization ability.

4.2.4. *Performance results*

In this section, we discuss the scalability of the mode in real-time network environments, for it is a crucial aspect of practical application.

Throughput. We truncate the packets to the first 200 bytes on the physical testbed and increase the sending rates until our model reaches maximum throughput. Our model achieves the maximum throughput

Table 4
Robustness analysis with $\lambda_{kl} = 0.032$ and $\lambda_{adv} = 0.95$.

Training data	Testing data	Accuracy	Precision	Recall	F1-Score
CICIDS 2017	CICIDS 2017	0.999	0.996	0.997	0.997
CICDDoS 2019	CICDDoS 2019	0.993	0.986	0.997	0.991
USTC-TFC 2016	USTC-TFC 2016	0.995	0.982	0.991	0.986
Synthetic Dataset	Synthetic Dataset	0.986	0.99	0.982	0.986
CICIDS 2017	CICDDoS 2019	0.745	0.830	0.709	0.765
CICIDS 2017	USTC-TFC 2016	0.845	0.843	0.839	0.841
CICDDoS 2019	USTC-TFC 2016	0.868	0.853	0.841	0.847
CICDDoS 2019	CICIDS 2017	0.887	0.914	0.867	0.890
CICDDoS 2019	CICIDS 2017	0.887	0.914	0.867	0.890
CICIDS 2017	Synthetic Dataset	0.863	0.906	0.834	0.868
CICIDS 2017+Algorithm 1	CICDDoS 2019	0.973	0.978	0.967	0.972
CICIDS 2017+Algorithm 1	USTC-TFC 2016	0.963	0.965	0.962	0.963
CICDDoS 2019+Algorithm 1	CICIDS 2017	0.971	0.975	0.964	0.972
CICDDoS 2019+Algorithm 1	USTC-TFC 2016	0.968	0.972	0.965	0.968
CICIDS 2017+Algorithm 1	Synthetic Dataset	0.954	0.960	0.948	0.954

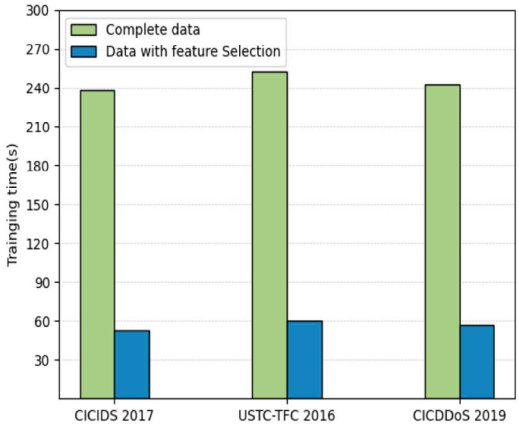


Fig. 8. Training time consumption with different features.

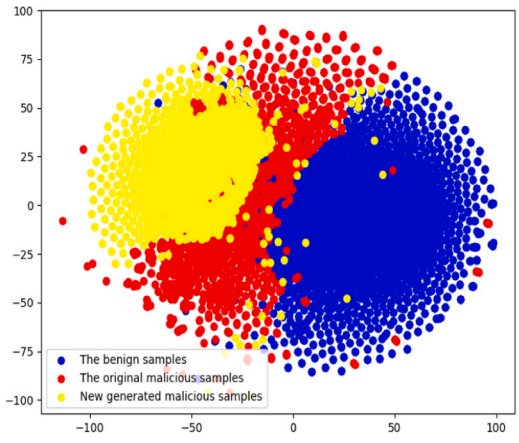


Fig. 9. Balanced data distribution plot, when using $\lambda_{kl} = 0.032$ and $\lambda_{adv} = 0.95$, the CICIDS 2017 dataset becomes more balanced (RF = 1:1), namely, the newly generated malicious samples and the original malicious samples equal to the benign samples.

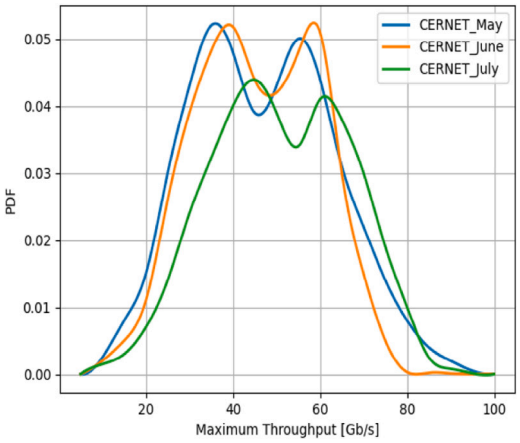


Fig. 10. Max throughput evaluation.

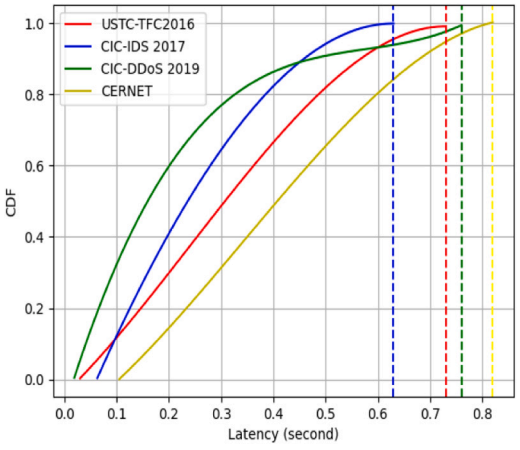


Fig. 11. Detection latency with different datasets.

(92.43) with the backbone traffic datasets. We observed continuously for three months and found it achieves an average of 38.43 ~ 45.71 Gb\s throughput and the maximum throughput of 83.72~ 92.43 Gb\s as shown in Fig. 10, where 'PDF' is the probability density function of the average throughput for short. The throughput on July 2023 is lower because College students have a holiday and have low original traffic volume.

Latency. We replay the four backbone network traffic datasets to measure their detection latency. The Cumulative Distribution Function (CDF) of the overall detection latency is shown in Fig. 11. The detection latency of our model is between 0.63 s and 0.82 s with different datasets, which shows that it can achieve real-time detection in high throughput networks. We also analyze the latency in each step. We find that 73.2% of the latency comes from the pre-training of Albert (i.e., 0.46 s on average). However, in the pre-training, Albert selected critical features for classification, enhancing the detection accuracy.

Resource Consumption. We finally evaluate the memory usage of our model, as shown in Fig. 12. Our model utilizes 1.73 GB of memory to maintain Albert's domain generalization and pre-training. Moreover, the memory consumption of the learning algorithm is 0.87~ 1.15 GB.

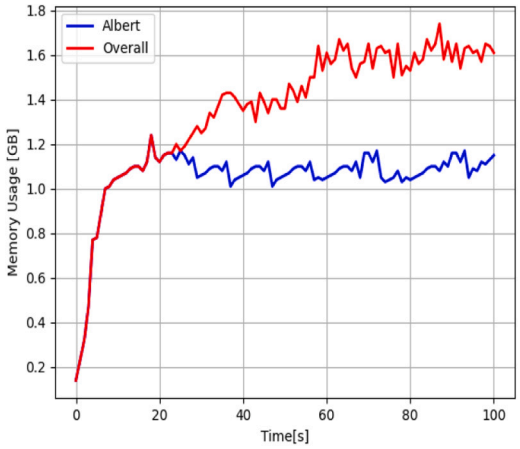


Fig. 12. Runtime memory usages.

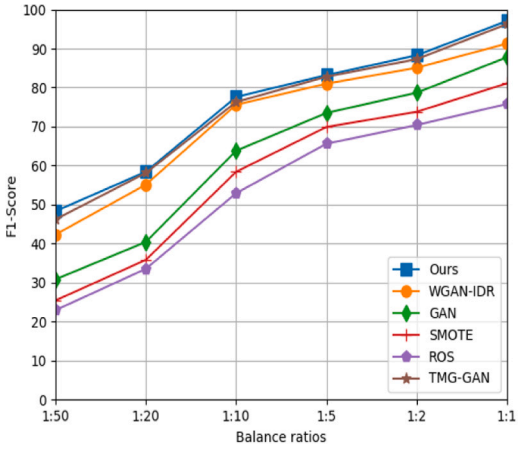


Fig. 13. Performance evaluation across various balance ratios.

Our model demonstrates that it retains more traffic information than other similar alternatives. Thus, the Albert and Deep Learning-based traffic analysis is more efficient than other algorithms.

4.3. Comparative experiments

We verify the superiority of our work with other studies employing machine learning or deep learning models for encrypted traffic classification and anomaly detection. They are [11,14,15,22,29,39,53]. We use "A1", "A2", "A3", "A4", "A5", "A6", and "A7" on behalf of each work for short, respectively, and give a brief introduction to make reads easy to follow.

[39] devised a plaintext-aware encrypted traffic detection framework to identify cobalt strike HTTPS traffic. It first parses handshake payloads into semantically explicit meta-features. Then, they utilized a transformer encoder to model the interaction between the attackers and victims and finally fused the interrelated metainformation and sequential information for prediction.

[11] proposed a natural language analysis model (TF-IDF) to identify malicious traffic. TF-IDF is commonly used in text feature analysis. Their work applied it to calculate the importance of keywords by analyzing the occurrence frequency (TF) of the keywords in the traffic. These important keywords can represent the traffic traits and effectively improve detection accuracy.

Table 5
Accuracy comparison based on the USTC-TFC 2016.

Algorithms	Overall Accuracy	Precision	Recall	F1-Score
Ours	99.9%	99.6%	99.7%	99.7%
A1	98.7%	98.1%	98.2%	98.1%
A7	98.3%	97.8%	98.1%	97.9%
A5	98.1%	97.6%	97.9%	97.7%
A4	97.8%	96.6%	96.2%	96.4%
A3	96.5%	96.1%	95.8%	95.9%
A6	96.4%	96.3%	95.4%	95.8%
A2	96.2%	95.8%	95.5%	95.6%

Table 6
Accuracy comparison based on the CICDDoS 2019.

Algorithms	Overall Accuracy	Precision	Recall	F1-Score
Ours	99.3%	98.6%	99.7%	99.1%
A7	98.5%	98.3%	97.7%	98.0%
A5	98.3%	98.1%	97.5%	97.8%
A1	97.8%	97.5%	97.2%	97.3%
A4	97.3%	97.1%	96.8%	96.9%
A6	97.3%	96.8%	96.7%	96.7%
A2	97.1%	96.9%	96.5%	96.7%
A3	96.8%	96.6%	96.2%	96.4%

[14] devised a flow-vector based detection scheme. A flow vector comprises the information in the packet header of the original traffic. They designed a word embedding method to make the semantics of the field value more meaningful and took advantage of the hierarchical attention mechanism to distinguish malicious traffic vectors.

[29] proposed a multi-session and multi-protocol-based scheme to solve the lack of sessions contextual information caused by many false negatives and false positives. They correlate multiple sessions together within different protocols to form session sequences. Experimental results demonstrate that their work has a high precision and recall rate.

[15] proposed regularized Wasserstein Generative Adversarial Networks to augment the minority attack samples and make a balanced dataset. This work is similar to part of ours, and we compare it to prove our work has better robust ability with nearly similar balanced data distribution.

[22] designed a causality explainable detection system to identify malicious encrypted traffic. The work increases the number of noncryptographic features and eliminates noise features to improve interpretability. They finally evaluated the performance through the detection balance and causal feature indexes.

[53] proposed data augmentation model TMG-GAN for intrusion detection. The data augmentation method is based on generative adversarial networks (GAN), which can generate different types of attack data. It calculated the cosine similarity between the generated samples and the original samples to improve the quality of the generated samples.

Firstly, we utilized different data augmentation methods to evaluate the data balance in the dataset. These baseline methods are Synthetic Minority Oversampling Technique (SMOTE), Generative Adversarial Network (GAN), Wasserstein GAN with Improved Deep Analytic Regularization (WGAN-IDR) [15], Tabular Multi-Generator Generative Adversarial Network (TMG-GAN) [53], and Random Over Sampling (ROS). Fig. 13 shows that our data augmentation method is closer to [53] but outperforms others. With increased BR, F1-Score is improved as more generated samples are trained. The increased trend of F1-scores demonstrates that the dataset becomes more balanced (BR = 1:1).

Then, we evaluated the detection accuracy with others based on the defined evaluation criteria of *Precision*, *Recall*, *Overall Accuracy*, and $F1 - Score$. In the comparison, we first use the same dataset: the training and test datasets are USTC-TFC 2016, CICDDoS 2019, and

Table 7
Accuracy comparison based on synthetic dataset.

Algorithms	Overall Accuracy	Precision	Recall	F1-Score
Ours	99.7%	99.4%	99.5%	99.4%
A5	98.7%	98.4%	98.2%	98.3%
A7	98.4%	98.2%	97.8%	98.0%
A4	97.5%	97.3%	97.1%	97.2%
A6	97.3%	97.1%	96.9%	97.0%
A3	97.1%	96.9%	96.7%	96.8%
A1	96.8%	96.9%	96.5%	96.7%
A2	96.6%	96.7%	96.4%	96.5%

Table 8
Robust evaluation with different dataset (Training dataset:CICIDS 2017 and Testing dataset:CICDDoS 2019).

Algorithms	Overall Accuracy	Precision	Recall	F1-Score
Ours	97.3%	97.8%	96.7%	97.2%
A7	93.2%	93.5%	92.6%	93.0%
A5	91.7%	92.1%	90.5%	91.3%
A1	85.7%	86.5%	84.1%	85.3%
A6	83.2%	84.1%	83.8%	83.9%
A3	81.6%	82.2%	82.4%	82.3%
A4	80.5%	79.8%	78.7%	79.2%
A2	74.5%	75.2%	71.8%	73.5%

Table 9
Robust evaluation with different dataset (Training dataset:CICDDoS 2019 and Testing dataset:synthetic dataset).

Algorithms	Overall Accuracy	Precision	Recall	F1-Score
Ours	97.3%	97.8%	96.7%	97.2%
A7	94.3%	93.8%	93.2%	93.5%
A5	92.5%	92.4%	91.7%	92.0%
A6	87.3%	87.6%	85.4%	86.5%
A1	83.8%	84.6%	84.2%	84.4%
A3	82.1%	83.2%	82.1%	82.6%
A4	80.2%	79.5%	78.2%	78.8%
A2	75.4%	75.8%	72.3%	74.0%

the synthetic dataset. Tables 5–7 evaluate the detection accuracy of different approaches.

We evaluate the robustness of different approaches with different datasets: the training dataset is CICIDS 2017, the test dataset is CICDDoS 2019, the training dataset is CICDDoS 2019, and the test dataset is the synthetic dataset. According to the data in Tables 8 and 9, the overall accuracy of our work improved by 22.8% and 21.9%, respectively. Our approach shows excellent performance in the encrypted malicious detection task and significantly improves by at most 22.8% compared to others not using domain generalization algorithms, such as 'A2', 'A4', 'A1', 'A6', and 'A3'. Although our data augmentation method is closer to [53], other indicators are superior, especially for robustness. Besides, we can find that our work applies to the Industrial Internet of Things (IIoT) environment. It can be deployed at edge nodes to provide network security support for IIoT devices.

In the experiments, we assume that attackers inject benign TLS, HTTP traffic, and UDP video traffic into the malicious traffic to disguise it for evasion. We use TLS, HTTP, and UDP video traffic because they account for a high proportion of benign traffic datasets,i.e., around 17%, 15%, and 12%, respectively. We mix the malicious traffic in the wild and the benign with a ratio of 1:1. We compare it with other works and observe that the attackers cannot evade our algorithm by injecting benign traffic into malicious traffic, as shown in Fig. 14. However, the attackers evade the detection by other approaches. We randomly select three malicious traffic cases for verification.

(1) webshell upload attack

Webshell is a backdoor trojan based on web service, and ASP, PHP, JSP, or CGI always write it. After hackers invade a website, they can

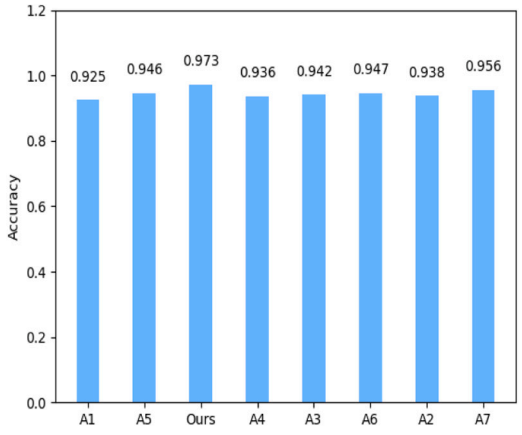


Fig. 14. Detection accuracy of attacks by injecting benign traffic.

hacker=%40eval%01%28base64_decode%28%24_POST%5Bz0%5D%29%29%3B&z0=QGluaV9zZXQoImRpc3BsYXlfZXJyb3JzIiwiMCIpO0BzZXRfdGltZV9saW1pdCgwKTtAc2V0X21hZ2ljX3F1b3Rlc19ydW50aW1lKDApO2VjaG8oi0%2BfCIpOzskcD1iYXNlNjRfZGVjb2RlKCRfUE9TVFsiejEiXSk7JHM9YmFzZTY0X

Fig. 15. Partial payload information of 122.*.*.10.

Port A	Port B	Packets	Bytes	Packets A→B	BytesA→B	Packets B→A	BytesB→A
32768	1036	1	74 bytes	1	74 bytes	0	0 bytes
32768	143	1	74 bytes	1	74 bytes	0	0 bytes
32768	81	1	74 bytes	1	74 bytes	0	0 bytes
32768	8180	1	74 bytes	1	74 bytes	0	0 bytes
32770	22	1	74 bytes	1	74 bytes	0	0 bytes
32770	43	1	74 bytes	1	74 bytes	0	0 bytes
32770	443	1	74 bytes	1	74 bytes	0	0 bytes
32770	458	1	74 bytes	1	74 bytes	0	0 bytes
32770	3306	1	74 bytes	1	74 bytes	0	0 bytes

Apache Shiro

Fig. 16. Partial port scan statistics of 211.*.*.193.

conduct file management, Command Execute, and use the browser to access the backdoor program to control the server. We found a webshell upload script in the traffic. The most crucial keyword is URI (Full request URI: http://122.*.*.10/upload/upload/shell.php), and the eigenvalue of it is 2.165. We analyze the original packet, as shown in Fig. 15. We found that the attacker used the eval function to deliver the attack payload and used base64_decode (POST[z0]) to decode the attack payload, where $z0$ may correspond to the received data by POST[z0]. We consider it a suspicious webshell upload attack with characteristics similar to China Chopper.

Privilege escalation on 211.*.*.193

The adversaries always use privilege escalation to gain higher-level permissions on a system or network. We have found a feature vector ([Packets Size, Cookie, Bytes Size, Source Port, Destination Port]), and their eigenvalue is [1.567,1.342,1.089,0.905,0.897]. IP (61.*.*.233) randomly scans multiple ports on destination host-B(211.*.*.193). We found that multiple ports in 211.*.*.193 are accessed. The bytes are less and have the same volume, as shown in Fig. 16. We also analyze the corresponding request and response packet for these ports and discover a malicious attempt at the Apache Shiro component. In the cookie of the request packet, the attacker assigns a random value to the $rememberMe$ variable. However, there is a $rememberMe = deleteMe$ field in the Set-Cookie value of the response packet. A base-64 encoded and AES-encrypted string exists in the $rememberMe$ field (e.g., $GOVxyFXOv2v0nEMsTHwZW900IX1$), which is the typical

Attack IP	OutBytes	InBytes	OutPkts	InPkts
0.0.*.148	13824	0	256	0
0.0.*.23	15360	0	256	0
0.0.*.219	15360	0	256	0
0.0.*.153	15360	0	256	0
0.0.*.228	13824	0	256	0
0.0.*.192	13824	0	256	0
0.0.*.138	15360	0	256	0
0.0.*.221	13824	0	256	0
0.0.*.244	15360	0	256	0
0.0.*.81	13824	0	256	0
0.0.*.253	15360	0	256	0
0.1.*.175	15360	0	256	0
0.1.*.243	13824	0	256	0
42.247.*.93	13824	0	256	0
42.247.*.98	13824	0	256	0

Fig. 17. Partial attack IP statistics of UDP Flood attack on 211.*.*.21.

traffic signature of the CVE-2016-4437. Therefore, we conclude that the attacker wants to privilege escalation by exploiting the Apache Shiro vulnerability on 211.*.*.193.

(3)UDP Flood attack on 211.*.*.21

In a UDP Flood attack, the attacker uses many zombie hosts, and all attack packets are directed to the victim or aggregated near the victim, resulting in the servers not working. We have found that 211.*.*.21 suffered UDP flood attacks at 10:15:22 on 3 Jun 2023, which lasted for five minutes. As there are 504,148 UDP packets, we adopted deep flow inspection and transformed them into flows for convenience. We found that the average attack intensity was close to 610MBPS, with over 1 million attack IPs. The attack host uses a random port to send meaningless UDP packets with the same byte (30B) to port 1025 of 211.*.*.21. Most IPs are fake and only used for internal network communication, such as 0.0.*.23, 0.1.*.175, and 42.247.*.98. We give partial statistics of the attack IPs in Fig. 17, where the "InPkts" and "OutPkts" indicates the total number of packets the victim received and sent, and the "InBytes" and "OutBytes" means the total number of bytes the victim received and sent.

Our work is robust and accurate, with low detection latency in the backbone. We only use the packet header information to obtain weighted important keywords representing traffic characteristics. All the feature extraction and selection processes are automatic, which is superior to previous works based on manual extraction. Besides, the devised adaptive domain generalization algorithm is robust against various datasets. It can augment the minority malicious samples to balance the dataset, which can adapt to different scenarios and be more suitable for deployment in the wild.

5. Conclusion and future work

This work devised a novel, robust approach for detecting encrypted malicious traffic based on the Albert and Deep Learning model. Our inspiration is that the packet header fields, as do the underlying grammatical rules for constructing sentences, have a strict order. We take the packet header information and capture important keywords to construct the characteristic representation of the traffic. We first devise an adaptive domain generalization algorithm with a new loss function. It can augment the minority malicious encrypted traffic to balance the dataset. Besides that, it can enhance robust detection ability in different datasets. We then consider all packet headers as text and use the ALBERT model to convert the qualitative data in the packets into quantitative data. By devising the feature selection algorithm, we obtained the best-weighted feature subset, which is more beneficial for encrypted malicious traffic detection with a 1D-CNN model. We have conducted many experiments and have deployed our system on CERNET. Furthermore, our work applies to the Industrial Internet of Things (IIoT) environment. It can be deployed at edge nodes to provide network security support for IIoT devices. Experimental results demonstrate that our work outperforms other state-of-the-art alternatives when using domain generalization for data augmentation, and the detection accuracy achieves at most 22.8% improvement compared to others not using the data augmentation algorithm.

Briefly, there are three directions in our future work to improve encrypted malicious traffic detection. First, our detection methods are supervised and rely on prior knowledge, which makes detecting unknown encrypted malicious traffic patterns inefficient. Therefore, we plan to design a novel unsupervised graph-learning-based detection scheme to identify unknown encrypted traffic patterns in response to malware evolution by analyzing the graph's connectivity sparsity features. Second, we plan to explore incremental learning and semantic integration techniques to enhance the deep learning model's interpretability without sacrificing accuracy. Third, most studies did not report time and resource utilization, which questions the applicability of the proposed NLP-based NIDS in the IIoT. For securing critical infrastructure, analyzing the trade-off between detection performance and computation performance while devising an NIDS model to achieve maximum detection rate with a lower false alarm rate at minimum processing time and cost is another direction of our future work.

CRediT authorship contribution statement

Xiaodong Zang: Writing – original draft, Methodology. **Tongliang Wang:** Validation. **Xinchang Zhang:** Formal analysis. **Jian Gong:** Formal analysis. **Peng Gao:** Writing – review & editing. **Guowei Zhang:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This research is support by Key Laboratory of Computer Network and Information Integration (Ministry of Education) , and the projects of the National Natural Science Foundation of China under Grant (No. 62072252) and Shandong Provincial Key Laboratory of Computer Networks, China under grant (No. ZR2021QF090).

References

[1] A. Shahraki, M. Abbasi, A. Taherkordi, A.D. Jurcut, A comparative study on online machine learning techniques for network traffic streams analysis, Comput. Netw. 207 (2022) 108836, http://dx.doi.org/10.1016/j.comnet.2022.108836.

[2] M. Abbasi, A. Shahraki, A. Taherkordi, Deep learning for Network Traffic Monitoring and Analysis (NTMA): A survey, Comput. Commun. 170 (2021) 19–41, http://dx.doi.org/10.1016/j.comcom.2021.01.021.

[3] J. Zhao, X. Jing, Z. Yan, W. Pedrycz, Network traffic classification for data fusion: A survey, Inf. Fusion 72 (2021) 22–47, http://dx.doi.org/10.1016/j.inffus.2021.02.009.

[4] Google, HTTPS encryption on the web, 2023, https://transparencyreport.google.com/https/overview.

[5] M.S. Raza, S.B.A. Kazmi, R. Ali, M.M. Naqvi, H. Fiaz, A. Akram, High performance DPI engine design for network traffic classification, metadata extraction and data visualization, in: 2024 5th International Conference on Advancements in Computational Sciences, ICACS, 2024, pp. 1–6, http://dx.doi.org/10.1109/ICACS60934.2024.10473274.

[6] P. Zhang, F. He, H. Zhang, J. Hu, X. Huang, J. Wang, X. Yin, H. Zhu, Y. Li, Real-time malicious traffic detection with online isolation forest over SD-WAN, IEEE Trans. Inf. Forensics Secur. 18 (2023) 2076–2090, http://dx.doi.org/10.1109/TIFS.2023.3262121.

[7] J. Holland, P. Schmitt, N. Feamster, P. Mittal, New directions in automated traffic analysis, in: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, CCS '21, Association for Computing Machinery, 2021, pp. 3366–3383, http://dx.doi.org/10.1145/3460120.3484758.

[8] Y. Hong, Q. Li, Y. Yang, M. Shen, Graph based encrypted malicious traffic detection with hybrid analysis of multi-view features, Inform. Sci. 644 (2023) 119229, http://dx.doi.org/10.1016/j.ins.2023.119229.

[9] Z. Wang, V.L. Thing, Feature mining for encrypted malicious traffic detection with deep learning and other machine learning algorithms, Comput. Secur. 128 (2023) 103143, http://dx.doi.org/10.1016/j.cose.2023.103143.

[10] Y. Fang, K. Li, R. Zheng, S. Liao, Y. Wang, A communication-channel-based method for detecting deeply camouflaged malicious traffic, Comput. Netw. 197 (2021) 108297, http://dx.doi.org/10.1016/j.comnet.2021.108297.

[11] H. Yang, Q. He, Z. Liu, Q. Zhang, Malicious encryption traffic detection based on NLP, Secur. Commun. Netw. 2021 (2021) 1–10.

[12] Z. Wang, K.W. Fok, V.L. Thing, Machine learning for encrypted malicious traffic detection: Approaches, datasets and comparative study, Comput. Secur. 113 (2022) 102542, http://dx.doi.org/10.1016/j.cose.2021.102542.

[13] B. Xu, G. He, H. Zhu, ME-Box: A reliable method to detect malicious encrypted traffic, J. Inform. Secur. Appl. 59 (2021) 102823, http://dx.doi.org/10.1016/j.jisa.2021.102823.

[14] J. Hou, F. Liu, H. Lu, Z. Tan, X. Zhuang, Z. Tian, A novel flow-vector generation approach for malicious traffic detection, J. Parallel Distrib. Comput. 169 (2022) 72–86, http://dx.doi.org/10.1016/j.jpdc.2022.06.004.

[15] R. Chapaneri, S. Shah, Enhanced detection of imbalanced malicious network traffic with regularized generative adversarial networks, J. Netw. Comput. Appl. 202 (2022) 103368, http://dx.doi.org/10.1016/j.jnca.2022.103368.

[16] Y. Zhao, B. Cui, J. Yang, M. Jiang, A DPI-based network traffic feature vector optimization model, in: L. Barolli (Ed.), Advances in Internet, Data & Web Technologies, Springer Nature Switzerland, Cham, 2024, pp. 522–531.

[17] H. Yan, H. Li, M. Xiao, R. Dai, X. Zheng, X. Zhao, F. Li, PGSM-DPI: Precisely guided signature matching of deep packet inspection for traffic analysis, in: 2019 IEEE Global Communications Conference, GLOBECOM, 2019, pp. 1–6, http://dx.doi.org/10.1109/GLOBECOM38437.2019.9013941.

[18] Q. Cheng, C. Wu, H. Zhou, D. Kong, D. Zhang, J. Xing, W. Ruan, Machine learning based malicious payload identification in software-defined networking, J. Netw. Comput. Appl. 192 (2021) 103186, http://dx.doi.org/10.1016/j.jnca.2021.103186.

[19] W. Niu, Z. Zhuo, X. Zhang, X. Du, G. Yang, M. Guizani, A heuristic statistical testing based approach for encrypted network traffic identification, IEEE Trans. Veh. Technol. 68 (4) (2019) 3843–3853, http://dx.doi.org/10.1109/TVT.2019.2894290.

[20] M. Nakahara, N. Okui, Y. Kobayashi, Y. Miyake, Machine learning based Malware traffic detection on IoT devices using summarized packet data, 2020, pp. 78–87, http://dx.doi.org/10.5220/0009345300780087.

[21] Y. Chen, J. Yang, S. Cui, C. Dong, B. Jiang, Y. Liu, Z. Lu, Unveiling encrypted traffic types through hierarchical network characteristics, Comput. Secur. 138 (2024) 103645, http://dx.doi.org/10.1016/j.cose.2023.103645.

[22] Z. Zeng, P. Xun, W. Peng, B. Zhao, Toward identifying malicious encrypted traffic with a causality detection system, J. Inform. Secur. Appl. 80 (2024) 103644, http://dx.doi.org/10.1016/j.jisa.2023.103644.

[23] L. Chen, S. Gao, B. Liu, Z. Lu, Z. Jiang, THS-IDPC: A three-stage hierarchical sampling method based on improved density peaks clustering algorithm for encrypted malicious traffic detection, J. Supercomput. 76 (2020) 7489–7518, http://dx.doi.org/10.1007/s11227-020-03372-1.

[24] I. Hafeez, M. Antikainen, A.Y. Ding, S. Tarkoma, IoT-KEEPER: Detecting malicious IoT network activity using online traffic analysis at the edge, IEEE Trans. Netw. Serv. Manag. 17 (1) (2020) 45–59, http://dx.doi.org/10.1109/TNSM.2020.2966951.

[25] Z. Fu, M. Liu, Y. Qin, J. Zhang, Y. Zou, Q. Yin, Q. Li, H. Duan, Encrypted Malware traffic detection via graph-based network analysis, in: Proceedings of the 25th International Symposium on Research in Attacks, Intrusions and Defenses, RAID '22, Association for Computing Machinery, 2022, pp. 495–509, http://dx.doi.org/10.1145/3545948.3545983.

[26] C. Fu, Q. Li, M. Shen, K. Xu, Realtime Robust Malicious Traffic Detection via Frequency Domain Analysis, CCS '21, Association for Computing Machinery, New York, NY, USA, 2021, pp. 3431–3446, http://dx.doi.org/10.1145/3460120.3484585.

[27] Z. Niu, J. Xue, D. Qu, Y. Wang, J. Zheng, H. Zhu, A novel approach based on adaptive online analysis of encrypted traffic for identifying Malware in IIoT, Inform. Sci. 601 (2022) 162–174, http://dx.doi.org/10.1016/j.ins.2022.04.018.

[28] K. Lin, X. Xu, F. Xiao, MFFusion: A multi-level features fusion model for malicious traffic detection based on deep learning, Comput. Netw. 202 (2022) 108658, http://dx.doi.org/10.1016/j.comnet.2021.108658.

[29] J. Liu, Q. Xiao, L. Xin, Q. Wang, Y. Yao, Z. Jiang, M3F: A novel multi-session and multi-protocol based Malware traffic fingerprinting, Comput. Netw. 227 (2023) 109723, http://dx.doi.org/10.1016/j.comnet.2023.109723.

[30] G. Apruzzese, M. Andreolini, M. Marchetti, A. Venturi, M. Colajanni, Deep reinforcement adversarial learning against botnet evasion attacks, IEEE Trans. Netw. Serv. Manag. 17 (4) (2020) 1975–1987, http://dx.doi.org/10.1109/TNSM.2020.3031843.

[31] F. Folino, G. Folino, M. Guarascio, F. Pisani, L. Pontieri, On learning effective ensembles of deep neural networks for intrusion detection, Inf. Fusion 72 (2021) 48–69, http://dx.doi.org/10.1016/j.inffus.2021.02.007.

[32] Q. Yuan, C. Liu, W. Yu, Y. Zhu, G. Xiong, Y. Wang, G. Gou, BoAu: Malicious traffic detection with noise labels based on boundary augmentation, Comput. Secur. 131 (2023) 103300, http://dx.doi.org/10.1016/j.cose.2023.103300.

[33] T.-L. Huoh, Y. Luo, P. Li, T. Zhang, Flow-based encrypted network traffic classification with graph neural networks, IEEE Trans. Netw. Serv. Manag. 20 (2) (2023) 1224–1237, http://dx.doi.org/10.1109/TNSM.2022.3227500.

[34] Z. Amiri, A. Heidari, N.J. Navimipour, M. Unal, A. Mousavi, Adventures in data analysis: A systematic review of deep learning techniques for pattern recognition in cyber-physical-social systems, Multim. Tools Appl. 83 (2023) 22909–22973, URL https://api.semanticscholar.org/CorpusID:260793874.

[35] A. Heidari, M.A.J. Jamali, Internet of Things intrusion detection systems: A comprehensive review and future directions, Cluster Comput. 26 (2022) 3753–3780, URL https://api.semanticscholar.org/CorpusID:253026443.

[36] A. Heidari, N.J. Navimipour, M. Unal, A secure intrusion detection platform using blockchain and radial basis function neural networks for internet of drones, IEEE Internet Things J. 10 (2023) 8445–8454, URL https://api.semanticscholar.org/CorpusID:256150756.

[37] A. Heidari, N.J. Navimipour, M.A.J. Jamali, S. Akbarpour, A green, secure, and deep intelligent method for dynamic IoT-edge-cloud offloading scenarios, Sustain. Comput. Inform. Syst. 38 (2023) 100859, URL https://api.semanticscholar.org/CorpusID:257121522.

[38] P. Luo, J. Chu, G. Yang, IP packet-level encrypted traffic classification using machine learning with a light weight feature engineering method, J. Inform. Secur. Appl. 75 (2023) 103519, http://dx.doi.org/10.1016/j.jisa.2023.103519.

[39] X. Yang, S. Ruan, Y. Yue, B. Sun, PETNet: Plaintext-aware encrypted traffic detection network for identifying Cobalt strike HTTPS traffics, Comput. Netw. 238 (2024) 110120, http://dx.doi.org/10.1016/j.comnet.2023.110120.

[40] A. Rukhin, J. Soto, J. Nechvatal, M. Smid, E. Barker, S. Leigh, M. Levenson, M. Vangel, D. Banks, A. Heckert, et al., A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications, vol. 22, US Department of Commerce, Technology Administration, National Institute of, 2001.

[41] J. Wang, C. Lan, C. Liu, Y. Ouyang, T. Qin, W. Lu, Y. Chen, W. Zeng, P.S. Yu, Generalizing to unseen domains: A survey on domain generalization, IEEE Trans. Knowl. Data Eng. 35 (8) (2023) 8052–8072, http://dx.doi.org/10.1109/TKDE.2022.3178128.

[42] Z.T. Sworna, Z. Mousavi, M.A. Babar, NLP methods in host-based intrusion detection systems: A systematic review and future directions, J. Netw. Comput. Appl. 220 (2023) 103761, http://dx.doi.org/10.1016/j.jnca.2023.103761, URL https://www.sciencedirect.com/science/article/pii/S1084804523001807.

[43] X. Zhang, Y. Ma, An ALBERT-based TextCNN-Hatt hybrid model enhanced with topic knowledge for sentiment analysis of sudden-onset disasters, Eng. Appl. Artif. Intell. 123 (2023) 106136, http://dx.doi.org/10.1016/j.engappai.2023.106136.

[44] D. Kim, P. Kang, Cross-modal distillation with audio–text fusion for fine-grained emotion classification using BERT and Wav2vec 2.0, Neurocomputing 506 (2022) 168–183, http://dx.doi.org/10.1016/j.neucom.2022.07.035.

[45] G. Ansari, T. Ahmad, M. Doja, Hybrid filter–wrapper feature selection method for sentiment classification, Arab. J. Sci. Eng. 44 (2019) 9191–9208, http://dx.doi.org/10.1007/s13369-019-04064-6.

[46] X. Li, H. Xiong, X. Li, X. Wu, X. Zhang, J. Liu, J. Bian, D. Dou, Interpretable deep learning: Interpretation, interpretability, trustworthiness, and beyond, Knowl. Inf. Syst. 64 (12) (2022) 3197–3234.

[47] S. Kiranyaz, O. Avci, O. Abdeljaber, T. Ince, M. Gabbouj, D.J. Inman, 1D convolutional neural networks and applications: A survey, Mech. Syst. Signal Process. 151 (2021) 107398, http://dx.doi.org/10.1016/j.ymssp.2020.107398.

[48] R. Abada, A.M. Abubakar, M.T. Bilal, An overview on deep leaning application of big data, Mesopotamian J. Big Data 2022 (2022) 31–35.

[49] M. Kravchik, A. Shabtai, Efficient cyber attack detection in industrial control systems using lightweight neural networks and PCA, IEEE Trans. Dependable Secure Comput. 19 (4) (2022) 2179–2197, http://dx.doi.org/10.1109/TDSC.2021.3050101.

[50] University of New Brunswick, Open-source evaluation dataset, 2023, https://www.unb.ca/cic/datasets/ddos-2019.html.

[51] X. Zang, J. Gong, M. Wang, P. Gao, G. Zhang, IP traffic behavior characterization via semantic mining, J. Netw. Comput. Appl. 213 (2023) 103603, http://dx.doi.org/10.1016/j.jnca.2023.103603, URL https://www.sciencedirect.com/science/article/pii/S108480452300022X.

[52] VirusTotal, Online virus detection tool, 2024, https://www.virustotal.com/gui/home/upload.

[53] H. Ding, Y. Sun, N. Huang, Z. Shen, X. Cui, TMG-GAN: Generative adversarial networks-based imbalanced learning for network intrusion detection, IEEE Trans. Inf. Forensics Secur. 19 (2024) 1156–1167.

Xiaodong Zang received his Ph.D. in School of Cyber Science and Engineering, Southeast University, Nanjing, China in 2020. He is a post doctor at the college of Electronic Optical Engineering, Nanjing University of Posts and Telecommunications University, Nanjing, China. Since 2020, he has been a lecturer in the school of Cyber Science of Engineering, Qufu Normal University. His research interests include computer networks and security, intrusion detection, network traffic and host profiling.

Tongliang Wang received his B.S. degree in computer software from Qufu Normal University, Qufu, China, in 2020. He is currently pursuing the M.D. degree in Qufu Normal University, Qufu, China. His current research focuses on computer networks and security, intrusion detection.

Xinchang Zhang (Senior Member, IEEE) received the Ph.D. degree from the Computer Network Information Center, Chinese Academy of Sciences, China, in 2010. He is currently a Professor at the Qilu University of Technology (Shandong Academy of Sciences). He has over 40 papers in research journals, such as IEEE Journal on Selected Areas in Communications, IEEE Transactions on Services Computing, IEEE Transactions on Vehicular Technology,etc.. His research interests include network protocols and architectures, and cloud computing.

Jian Gong is a professor in the School of Cyber Science and Engineering, Southeast University. His research interests are network architecture, network intrusion detection, and network management. He has received his B.S. in computer software from Nanjing University, and his Ph.D. in computer science and technology from Southeast University.

Peng Gao received the Ph.D. degree from the Harbin Institute of Technology, China, in 2020. He is currently an Associate Professor with the School of Cyber Science and Engineering, Qufu Normal University, China. His current research interests include signal and image processing, deep learning and computer vision. He serves as a reviewer of top journals such as the IEEE Transactions on Circuits and Systems for Video Technology, Information Sciences, and IET Image Processing.

Guowei Zhang received the B.S. degree from the Huazhong University of Science and Technology, Wuhan, China, in 2014, and the Ph.D. degree in communication and information systems from the University of Chinese Academy of Sciences, Beijing, China, in 2019. Since 2020, he has been a lecturer in the school of Cyber Science of Engineering, Qufu Normal University. His current research interests include the performance optimization and privacy protection in intelligent computing networks.